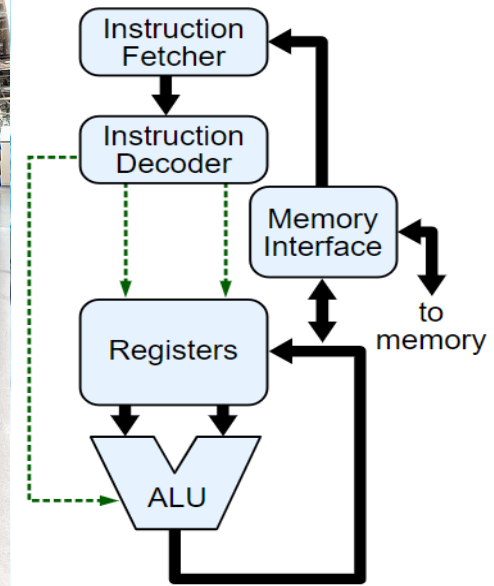
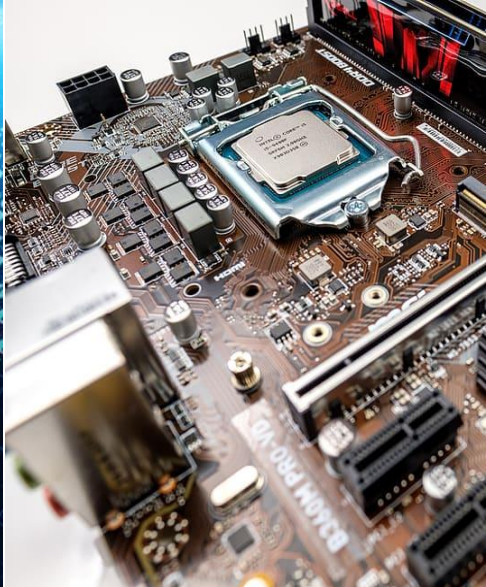




What does the following mean?

32-bits

```
0010011110111101111111111111100000 Instruction 1
10101111101111111000000000000010100 Instruction 2
10101111101001000000000000000100000
10101111101001010000000000000100100
1010111110100000000000000000011000
1010111110100000000000000000011100
1000111110101110000000000000011100
1000111110111000000000000000011000
0000000111001110000000000000011001
00100101110010000000000000000001
00101001000000010000000001100101
1010111110101000000000000000011100
0000000000000000000111100000010010
00000011000011111100100000100001
00010100001000001111111111110111
1010111110111001000000000000011000
00111100000001000001000000000000
1000111110100101000000000000011000
000011000001000000000000011101100
00100100100001000000010000110000
1000111110111111000000000000010100
001001111011110100000000000100000
000000111110000000000000000001000
000000000000000000001000000100001
```

**MIPS machine language
code for a routine to
compute and print the sum
of the squares of integers
between 0 and 100.**

Assemblers,
Linkers,
and the SPIM
Simulator

James R. Larus

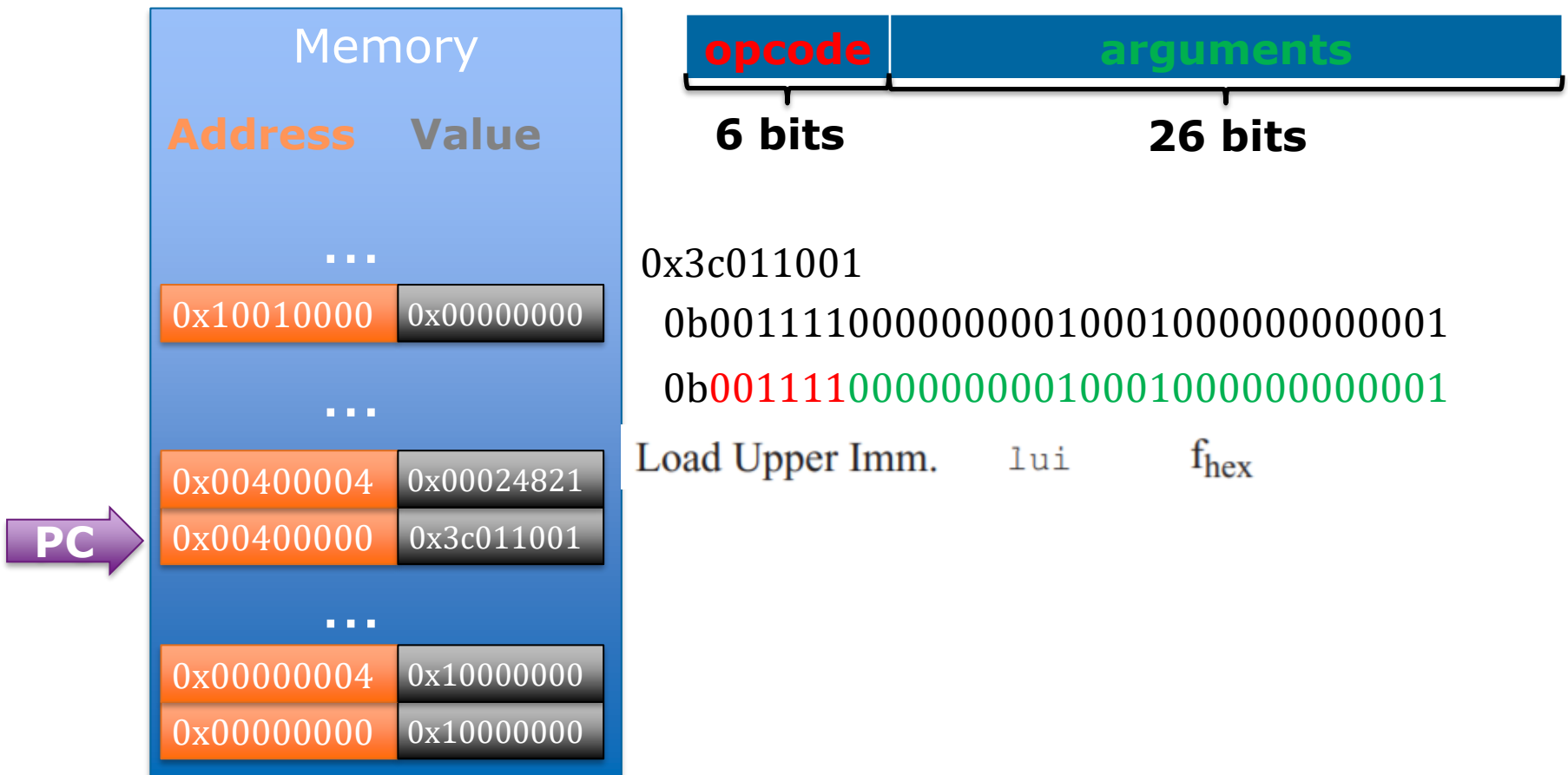
Microsoft Research

MIPS Registers

- ▶ The MIPS32 CPU has 32 registers: \$0, ..., \$31, each of size 32 bits.
- ▶ Different registers serve different purposes (e.g., \$0 is *always* 0).
- ▶ Unlike x86 and other complex ISAs, MIPS only operate on registers.
- ▶ Simplifies the ISA, but increases #instructions.
- ▶ For example, `Mem[0x1001000] += Mem[0x1001004]` translates to:
 - Read `Mem[0x1001000]` into register.
 - Read `Mem[0x1001004]` into register.
 - Compute the sum of both registers.
 - Write the result back to `Mem[0x1001000]`.

MIPS – a RISC processor

- ▶ Having simple instructions means we have fewer of them.



Something to think about

- ▶ In MIPS32, addresses are 32-bit long.
- ▶ In MIPS32, instructions are 32-bit long.
 - And 6 bits out of these are used for opcode.
- ▶ How can we encode an address?
 - E.g., for jump location, or loading array address into a pointer.
- ▶ How can we set a 32-bit value into a register?

Instruction types

- ▶ Three instruction types:
 - **R**-ithmetic (between register) operations.
 - add \$8, \$1, \$2
 - sub \$12, \$6, \$3
 - **I**-mmediate (including fixed numbers) operations.
 - addi \$15, \$15, -1
 - beq \$10, \$14, 0x00000002
 - **J**-ump operations.
 - j 0x00400050
 - jal 0x0040007c

The Reference Card

- On the website.

separate card 2. Fold bottom side (columns 3 and 4) together

MIPS Reference Data

CORE INSTRUCTION SET

NAME, MNEMONIC	FOR-MAT	OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
Add	add R	$R[rd] = R[rs] + R[rt]$	(1) 0 / 20 _{hex}
Add Immediate	addi I	$R[rt] = R[rs] + \text{SignExtImm}$	(1,2) 8 _{hex}
Add Imm. Unsigned	addiu I	$R[rt] = R[rs] + \text{SignExtImm}$	(2) 9 _{hex}
Add Unsigned	addu R	$R[rd] = R[rs] + R[rt]$	0 / 21 _{hex}
And	and R	$R[rd] = R[rs] \& R[rt]$	0 / 24 _{hex}
And Immediate	andi I	$R[rt] = R[rs] \& \text{ZeroExtImm}$	(3) c _{hex}
Branch On Equal	beq I	if($R[rs] == R[rt]$) $PC = PC + 4 + \text{BranchAddr}$	(4) 4 _{hex}
Branch On Not Equal	bne I	if($R[rs] != R[rt]$) $PC = PC + 4 + \text{BranchAddr}$	(4) 5 _{hex}
Jump	j J	$PC = \text{JumpAddr}$	(5) 2 _{hex}
Jump And Link	jal J	$R[31] = PC + 8; PC = \text{JumpAddr}$	(5) 3 _{hex}
Jump Register	jr R	$PC = R[rs]$	0 / 08 _{hex}
Load Byte Unsigned	lbu I	$R[rt] = \{24'b0, M[R[rs] + \text{SignExtImm}](7:0)\}$	(2) 24 _{hex}
Load Halfword Unsigned	lhu I	$R[rt] = \{16'b0, M[R[rs] + \text{SignExtImm}](15:0)\}$	(2) 25 _{hex}
Load Linked	ll I	$R[rt] = M[R[rs] + \text{SignExtImm}]$	(2,7) 30 _{hex}
Load Upper Imm.	lui I	$R[rt] = \{\text{imm}, 16'b0\}$	f _{hex}

①



ARITHMETIC CORE INSTRUCTION SET

NAME, MNEMONIC	FOR-MAT	OPERATION	OPCODE / FUNCT (Hex)
Branch On FP True	bclt FI	if(FPcond) $PC = PC + 4 + \text{BranchAddr}$	(4) 11/8/1/--
Branch On FP False	bclt FI	if(!FPcond) $PC = PC + 4 + \text{BranchAddr}$	(4) 11/8/0/--
Divide	div R	$Lo = R[rs] / R[rt]; Hi = R[rs] \% R[rt]$	0/--/1a
Divide Unsigned	divu R	$Lo = R[rs] / R[rt]; Hi = R[rs] \% R[rt]$	(6) 0/--/1b
FP Add Single	add.s FR	$F[fd] = F[fs] + F[ft]$	11/10/--/0
FP Add Double	add.d FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} + \{F[ft], F[ft+1]\}$	11/11/--/0
FP Compare Single	c.x.s* FR	$FPcond = (F[fs] \text{ op } F[ft]) ? 1 : 0$	11/10/--/y
FP Compare Double	c.x.d* FR	$FPcond = (\{F[fs], F[fs+1]\} \text{ op } \{F[ft], F[ft+1]\}) ? 1 : 0$ * (x is eq, lt, or le) (op is ==, <, or <=) (y is 32, 3c, or 3e)	11/11/--/y
FP Divide Single	div.s FR	$F[fd] = F[fs] / F[ft]$	11/10/--/3
FP Divide Double	div.d FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} / \{F[ft], F[ft+1]\}$	11/11/--/3
FP Multiply Single	mul.s FR	$F[fd] = F[fs] * F[ft]$	11/10/--/2
FP Multiply Double	mul.d FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} * \{F[ft], F[ft+1]\}$	11/11/--/2
FP Subtract Single	sub.s FR	$F[fd] = F[fs] - F[ft]$	11/10/--/1
FP Subtract Double	sub.d FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} - \{F[ft], F[ft+1]\}$	11/11/--/1
Load FP Single	lwc1 I	$F[rt] = M[R[rs] + \text{SignExtImm}]$	(2) 31/--/--
Load FP Double	ldc1 I	$F[rt] = M[R[rs] + \text{SignExtImm}]; F[rt+1] = M[R[rs] + \text{SignExtImm} + 4]$	(2) 35/--/--
Move From Hi	mfhi R	$R[rd] = Hi$	0/--/--/10
Move From Lo	mflo R	$R[rd] = Lo$	0/--/--/12
Move From Control	mfc0 R	$R[rd] = CR[rs]$	10/0/--/0

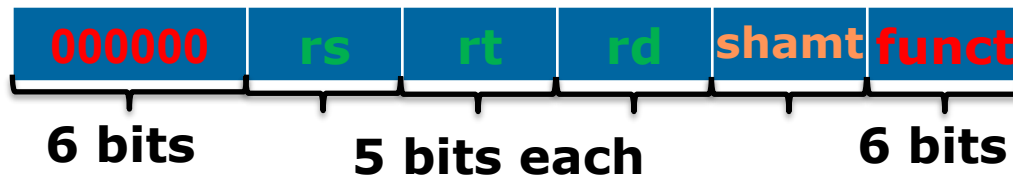
②

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, Computer Organization and Design, 4th ed.

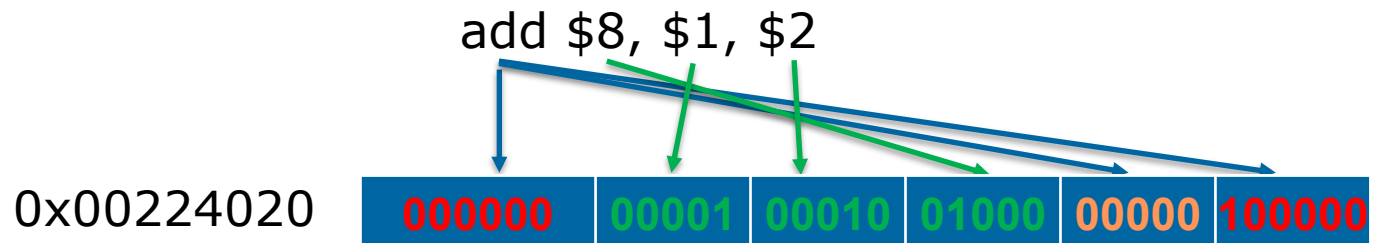
R-ithmetic instructions

- ▶ Opcode is 0.

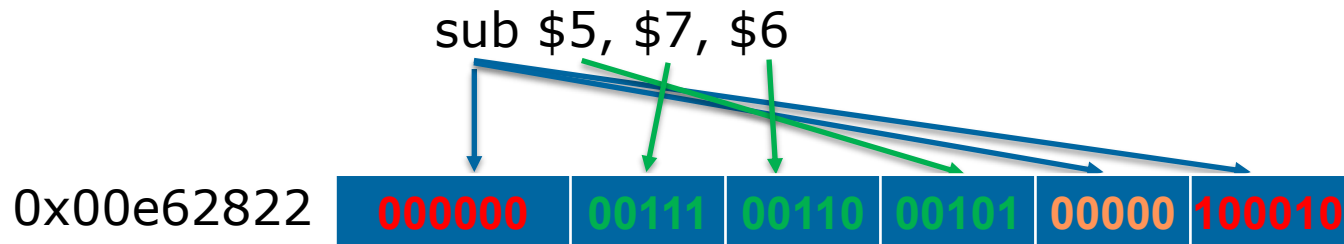
NAME, MNEMONIC	FOR- MAT	OPERATION (in Verilog)	/ FUNCT (Hex)
Add	add R	$R[rd] = R[rs] + R[rt]$	(1) 0 / 20 _{hex}



$R[rd] = \text{func}(R[rt], R[rs])$



R-ithmetic instructions



Subtract

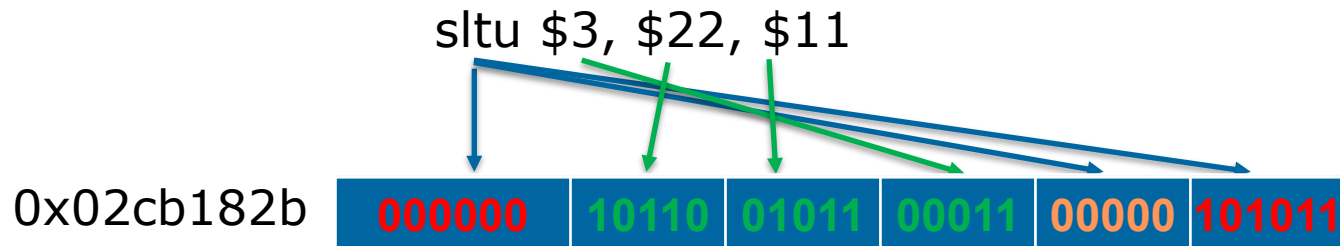
sub R $R[rd] = R[rs] - R[rt]$

(1) 0 / 22_{hex}

Set Less Than Unsig. sltu

R $R[rd] = (R[rs] < R[rt]) ? 1 : 0$

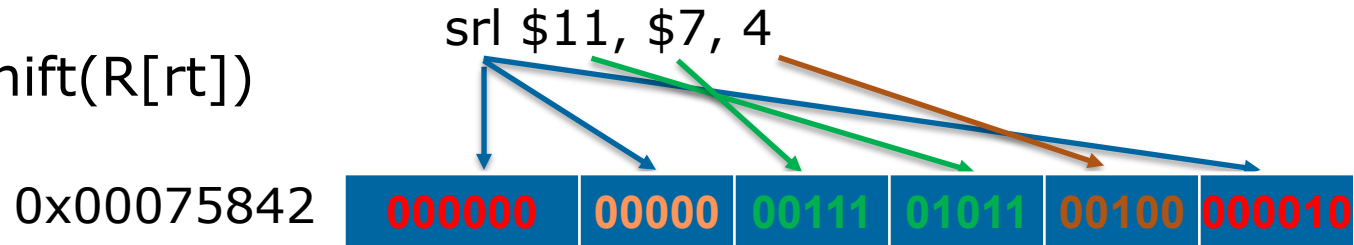
(6) 0 / 2b_{hex}



Shift instructions

- rs is 0.

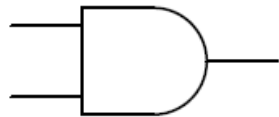
– $R[rd] = \text{shift}(R[rt])$



Shift Right (Logical) srl R $R[rd] = R[rt] \gg \text{shamt}$ 0 / 02_{hex}

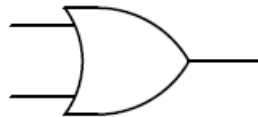
- $100_{10} \ll 4 = 100 \cdot 2^4 = 1600$ $1100100_2 \ll 4 = 1100100\text{0000}_2$
- $100_{10} \gg 4 = \left\lfloor \frac{100}{2^4} \right\rfloor = 6$ $1100100_2 \gg 4 = 110\text{0100}_2$
 $\text{0xffff9c} \gg 4 = \text{0xfffff9}$
 $= 268435449_{10}$
- $-100_{10} \gg 4 = ?$
 - Logical
 - Arithmetic $\left\lfloor \frac{-100}{2^4} \right\rfloor = -7$
 $= \text{0xfffff9c} \gg 4$

Bitwise operations



AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



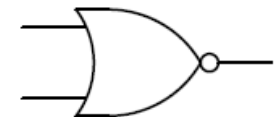
OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



NOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

- ▶ Apply bit by bit (bit-wise).

and \$9, \$4, \$5

\$4 = 11010000101011010001111110100000

\$5 = 00101111010100001010000010001111

\$9 = 0000000000000000000000000000000010000000

Pseudo instructions

- ▶ *E.g.*, move \$8, \$9
- ▶ *Not* real MIPS32 instructions!
 - Have *no* opcodes and are not recognized by the CPU.
- ▶ *Are* translated into real instructions by the assembler.
 - move \$8, \$9 can be translated into add \$8, \$9, \$0.

Pseudo instructions

Lecture 2: the NOT pseudo-instruction



Pseudo instructions: $\$8 \leftarrow \sim \9



sub \$8,
\$0, \$9

-\$8) is not the
same as ~(\$8)

0000000000000000
0000000000000000 = 0

1111111111111111
1111111111111111 = -1



nor \$8,
\$9, \$9

$\$9 \mid \$9 = \$9$



nand \$8,
\$9, \$9

There is no
such
instruction!



nor \$9,
\$0, \$8

Wrong
parameter
order



nor \$8,
\$9, \$0

$\$9 \mid 0..0 = \9

MARS's
implementation

High-level commands

► $x = a - b + c - d;$

High level language

Not unique!

Assembly

Is unique!

Machine code

Reverse engineering

Disassembly

```
sub    $10, $4, $5
sub    $11, $6, $7
add    $12, $10, $11
```

```
sub    $10, $4, $5
sub    $10, $10, $7
add    $12, $10, $6
```

```
0x00855022
0x00C75822
0x014B6020
```

Examples of R-Type Instructions

Instruction	op	funct (Base 16)	Description
add	0	0x20	Addition – with signed overflow
sub	0	0x22	Subtraction – with signed overflow
and	0	0x24	Logical And
or	0	0x25	Logical Or
xor	0	0x26	Logical Xor
nor	0	0x27	Logical Nor
slt	0	0x2A	Set on less than
sltu	0	0x2B	Set on less than unsigned
srl	0	0x02	Shift Right Logical
sra	0	0x03	Shift Right Arith.
sll	0	0x00	Shift Left Logical